

Inteligencia Artificial

Búsqueda heurística en radares.

Wenjie Chen - 100522255

Daniel Deblas Payo - 100522144

Xiya Ye - 100522159

15 de mayo del 2025

Índice

Introducción.....	2
1. Modelado del problema de búsqueda.....	3
1.1. Modelado matemático de estados y operadores.....	3
1.2. Estado inicial y estado final.....	3
1.3. Heurísticas admisibles.....	4
1.3.1. Heurística 1: Distancia Euclidiana Escalada.....	4
1.3.2. Heurística 2: Distancia de Manhattan Escalada.....	4
2. Generación del mapa y el espacio de búsqueda.....	5
2.1. Generación del mapa.....	5
2.2. Construcción del grafo de búsqueda.....	6
3. Integración del Algoritmo A*.....	7
4. Experimentos y batería de pruebas.....	8
Uso de la IA generativa.....	12
Conclusión.....	14



Introducción

En esta práctica, se aborda el diseño e implementación de un sistema de navegación para un avión espía que debe desplazarse sobre un área vigilada por radares, minimizando su probabilidad de detección.

El objetivo principal es desarrollar un mapa de probabilidades de detección basado en modelos físicos de radar y utilizar algoritmos de búsqueda heurística, como el algoritmo de A*, para planificar rutas seguras entre los distintos puntos de interés (POIs). La solución a este problema la hemos dividido en 3 puntos:

- Modelado matemático del espacio de búsqueda y las propiedades del radar.
- Generación de un grafo de navegación que representa las zonas de riesgo.
- Integración del algoritmo A* para la búsqueda de los caminos más óptimos.

Este documento detalla el proceso de implementación, desde la construcción del mapa hasta la ejecución de A*, validando su eficacia mediante diferentes experimentos en distintos escenarios, tanto los proporcionados como nuevos escenarios.

1. Modelado del problema de búsqueda

1.1. Modelado matemático de estados y operadores

El problema de navegación del avión espía se modela como un problema de búsqueda en grafo, donde cada nodo representa una posible posición del avión en una malla discreta, y las aristas representan movimientos válidos entre celdas adyacentes.

Siendo S , el espacio de estados, un estado $s \in S$ se define como $s = (i, j)$, donde i es el índice de fila en el malla ($i \in \{0, \dots, H-1\}$), j es el índice de columna en el malla ($j \in \{0, \dots, W-1\}$) y la dimensión de la malla es $H \times W$.

Definiendo A como las acciones, los operadores definen los movimientos posibles desde un estado $s = (i, j)$. Siendo $A(s) = \{\text{Norte}, \text{Sur}, \text{Este}, \text{Oeste}\}$, donde cada condición está condicionado por (teniendo en cuenta los límites del mapa):

- Norte(s): $(i-1, j)$ si $i > 0$
- Sur(s): $(i+1, j)$ si $i < H-1$
- Este(s): $(i, j+1)$ si $j < W-1$
- Oeste(s): $(i, j-1)$ si $j > 0$

1.2. Estado inicial y estado final

El estado inicial, s_0 depende del primer Punto de Interés (POI) en la secuencia de visita, que sería $s_0 = (i_0, j_0)$ donde (i_0, j_0) son las coordenadas del primer POI en la malla.

Como el problema consiste en visitar una secuencia de k puntos de intereses en un óptimo orden. Por tanto, se define un conjunto de estados meta:

$$F = \{f_1, f_2, \dots, f_{k-1}\}$$

donde cada f_m corresponde a las coordenadas del siguiente punto de interés en la secuencia. Entonces, podemos decir que la solución implica resolver una serie de subproblemas de búsqueda A^* entre cada par de POIs consecutivos:

$$A^*(\text{POI}_1 \rightarrow \text{POI}_2), A^*(\text{POI}_2 \rightarrow \text{POI}_3), \dots, A^*(\text{POI}_{k-1} \rightarrow \text{POI}_k)$$

1.3. Heurísticas admisibles

Una heurística $h(s)$ es admisible si nunca sobrestima el costo real $h^*(s)$ desde el estado s hasta la meta. Algunos diseños que hemos considerado implementar son la distancia de Manhattan Escalada y la de la Distancia Euclidiana Escalada. Por tanto, hemos de comprobar que ambas sean admisibles antes de la implementación.

1.3.1. Heurística 1: Distancia Euclidiana Escalada

Siendo la fórmula:

$$h_2(s) = \sqrt{(i - i_{\text{objetivo}})^2 + (j - j_{\text{objetivo}})^2} \cdot \epsilon$$

Podemos decir que esta heurística es admisible, puesto que la distancia euclidiana es siempre $\leq$ que la distancia de Manhattan, y al multiplicarlo por ϵ nos aseguramos que $h_2(s) \leq h^*(s)$.

En la implementación de esta heurística, al igual que en la heurística anterior, primero calculamos la distancia entre el nodo actual y el nodo objetivo. Por último, multiplicamos el resultado de la suma por epsilon para ajustar el valor.

Por lo que podemos concluir que ambas heurísticas propuestas cumplen con los siguientes puntos:

- No sobrestiman el costo real: $h(s) \leq h^*(s)$, $\forall s \in S$
- Se basan en distancias geométricas mínimas, considerando el peor caso de costo mínimo (ϵ).

1.3.2. Heurística 2: Distancia de Manhattan Escalada

La fórmula de ésta es:

$$h_1(s) = (|i - i_{\text{objetivo}}| + |j - j_{\text{objetivo}}|) \cdot \epsilon$$

Esta heurística es admisible debido a que la distancia de Manhattan se trata de un límite inferior del costo real, ya que asume movimiento directo sin obstáculos. Además, teniendo en cuenta que ϵ es el valor mínimo de probabilidad en el mapa, por lo que nos garantiza que $h_1(s) \leq h^*(s)$.

Para la implementación de esta heurística, hemos usado la fórmula para calcular la distancia. Para ello, calculamos la diferencia entre las coordenadas del nodo actual y el nodo objetivo. Tras obtener estos datos, calculamos la distancia con la fórmula correspondiente. Por último, se multiplica este valor por EPSILON para ajustar el resultado.

2. Generación del mapa y el espacio de búsqueda

2.1. Generación del mapa

El mapa de detección lo hemos construido como una matriz bidimensional donde cada celda (i, j) almacena la posibilidad de que el avión sea detectado al sobrevolar esa posición del mapa.

Para ello, hacemos primero el cálculo de la posibilidad de detección. Para cada radar R_k ubicado en (lat_k, lon_k) , hemos de calcular su influencia en todas las celdas del mapa mediante una función gaussiana bidimensional:

$$\Psi(x_1, x_2, \dots, x_n) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} e^{-\frac{1}{2}(x - \mu)^T \Sigma^{-1} (x - \mu)}$$

Para la implementación, hemos considerado las siguientes ecuaciones que nos facilitaba el enunciado:

- La ecuación 3 para la combinación de múltiples radares, ya que para cada celda, se selecciona el valor máximo de entre todos los radares.

$$\Psi^*(lat, lon) = \max_{\Psi_i} \{\Psi_i^*(lat, lon)\}_{i=1}^{N_r}$$

- La ecuación 1 y 4 para el cálculo del umbral de distancia máxima, puesto que si la celda está fuera del alcance R_{max} del radar, su contribución es nula.

$$R_{max} = \left(\frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 P_{min} L} \right)^{\frac{1}{4}} \quad \Psi_i^*(lat, lon) = \begin{cases} \Psi_i(lat, lon) & \text{si } d \leq R_{max} \\ 0 & \text{si } d > R_{max} \end{cases}$$

- La ecuación 8 para la normalización de probabilidades. Así, los valores crudos de posibilidad se escalan al intervalo $[\epsilon, 1]$ mediante la transformación, donde ϵ es un valor pequeño (10^{-5}) para así evitar probabilidades nulas, facilitando el diseño de heurísticas admisibles.

$$\Psi_{scaled}^* = \frac{\Psi^* - \Psi_{min}^*}{\Psi_{max}^* - \Psi_{min}^*} (1 - \epsilon) + \epsilon$$



En cuanto a la generación del mapa, hemos tenido que implementar un método en la clase Map que calcule el mapa de detección para toda la malla del terreno teniendo en cuenta los radares.

Primero, se inicializa una matriz de ceros con el mismo tamaño que el mapa. Luego, se generan los valores de latitud y longitud para cada fila y columna. A continuación, se recorre cada celda del mapa, obteniendo sus coordenadas geográficas, y se acumula el nivel de detección que aportan todos los radares sobre esa celda. Finalmente, se aplica una normalización para escalar los valores de la matriz al rango $[\text{EPSILON}, 1]$, asegurando que ninguna celda tenga un valor exactamente cero. El resultado es una matriz que presenta la probabilidad de detección en cada celda del mapa.

2.2. Construcción del grafo de búsqueda

El espacio de navegación se modela como un grafo dirigido ponderado, $G = (V, E)$, para ello lo implementamos mediante la biblioteca *networkx*.

En cuanto a la estructura del grafo, podemos distinguir entre nodos y aristas. Cada celda del mapa se convierte en un nodo, almacenando su probabilidad de detección como atributo. Y entre ellos, se crean conexiones (aristas) a celdas adyacentes, solo si la celda destino está dentro de los límites del mapa y si su probabilidad no supera el umbral de tolerancia especificado.

Para la asignación de pesos, como las aristas se ponderan de manera asimétrica, dada una arista, su peso es la probabilidad de la celda destino. Esto nos indica que el riesgo lo impone la posición a la que se llega y no la de partida.

Una vez construido el grafo, hemos implementado un método que calcule los costes que tiene un plan sobre el grafo. En primer lugar, se inicializa el coste total a 0. Después se recorre cada camino del plan de solución además de cada par de nodos consecutivos. Se comprueba que si el nodo está en formato cadena de texto para que si está de esa forma, se convierta en tupla. Por último se suma el peso de la arista al coste del plan y se devuelve el resultado al terminar.

3. Integración del Algoritmo A*

Por último, debemos ejecutar el algoritmo de A*. Para ello, utilizamos el algoritmo *astar_path* de *networkx*, que requiere de una función heurística y los pesos de aristas. Como tenemos implementadas ambas cosas, podemos iniciar con el proceso de búsqueda. Primero debemos discretizar las coordenadas de los puntos de interés, para que se conviertan en índices de la malla. Para ello, hemos implementado un método que se encarga solamente de eso.

Éste recibe las coordenadas a convertir, y las dimensiones del mapa. En primer lugar, con los valores que se corresponden al mapa, se crea una cuadrícula que representa el área del mapa. Tras ello, para todos los puntos del plan, se busca el índice x e y que se corresponde a la fila y la columna más cerca respectivamente. Después de esto, se añaden los puntos ya convertidos al plano.

Una vez con las coordenadas discretizadas, pasamos a la planificación de rutas. Como el orden de los nodos es escoger el más cercano, hemos decidido crear un método que busca el nodo más cercano. Primero, se inicializa una variable con coste infinito y otra para almacenar el nodo más cercano. Luego, se recorre cada nodo objetivo dentro de la lista de nodos candidatos. Para cada uno, se intenta calcular el coste del camino más corto desde el nodo del inicio usando *nx.shortest_path_length* con el peso asociado. Si el peso es menor que el mínimo actual, se actualiza el nodo más cercano y su coste. Si no existe camino entre 2 nodos, se ignora y se continúa. Al finalizar, se devuelve el nodo más cercano encontrado.

Cuando ya tenemos el nodo más cercano, podemos ejecutar A* sobre el grafo acumulando el coste total.

4. Experimentos y batería de pruebas

Escenarios	Variables	h1	h2
Escenario 0	Grafo	256 nodos y 913 aristas	256 nodos y 913 aristas
	Total path cost	0.24069947004318237	0.24069947004318237
	Number of expanded nodes	280	280
	Rango de nodos	(0,0) a (15,15)	(0,0) a (15,15)
	Coordenadas discretas	[(0, 0), (0, 10)]	[(0, 0), (0, 10)]
Escenario 1	Grafo	256 nodos y 903 aristas	256 nodos y 903 aristas
	Total path cost	1.0269875526428223	1.0269875526428223
	Number of expanded nodes	260	260
	Rango de nodos	(0,0) a (15,15)	(0,0) a (15,15)
	Coordenadas discretas	[(0, 0), (0, 10)]	[(0, 0), (0, 10)]
Escenario 2	Grafo	1024 nodos y 3307 aristas	1024 nodos y 3307 aristas
	Total path cost	7.4207329750061035	7.4207329750061035
	Number of expanded nodes	1261	1261
	Rango de nodos	(0,0) a (31,31)	(0,0) a (31,31)
	Coordenadas discretas	[(30, 3), (23, 7), (30, 15), (13, 21)]	[(30, 3), (23, 7), (30, 15), (13, 21)]
Escenario 3	Grafo	1024 nodos y 3253 aristas	1024 nodos y 3252 aristas
	Total path cost	11.07253360748291	11.07253360748291
	Number of expanded nodes	1267	1267
	Rango de nodos	(0,0) a (31,31)	(0,0) a (31,31)

	Coordenadas discretas	[(30, 3), (23, 7), (30, 15), (13, 21)]	[(30, 3), (23, 7), (30, 15), (13, 21)]
Escenario 4	Grafo	1024 nodos y 2969 aristas	1024 nodos y 2969 aristas
	Total path cost	6.683856964111328	6.683856964111328
	Number of expanded nodes	448	448
	Rango de nodos	(0,0) a (31,31)	(0,0) a (31,31)
	Coordenadas discretas	[(30, 3), (23, 7), (30, 15), (13, 21)]	[(30, 3), (23, 7), (30, 15), (13, 21)]
Escenario 5	Grafo	4096 nodos y 7764 aristas	4096 nodos y 7764 aristas
	Total path cost	23.43644905090332	23.43644905090332
	Number of expanded nodes	2217	2217
	Rango de nodos	(0,0) a (63,63)	(0,0) a (63,63)
	Coordenadas discretas	[(0, 33), (0, 60), (27, 60), (60, 53), (60, 31), (60, 15)]	[(0, 33), (0, 60), (27, 60), (60, 53), (60, 31), (60, 15)]
Escenario 6	Grafo	4096 nodos y 12550 aristas	4096 nodos y 12550 aristas
	Total path cost	28.12846565246582	28.12846565246582
	Number of expanded nodes	3560	3560
	Rango de nodos	(0,0) a (63,63)	(0,0) a (63,63)
	Coordenadas discretas	[(0, 33), (0, 60), (27, 60), (60, 53), (60, 31), (60, 15)]	[(0, 33), (0, 60), (27, 60), (60, 53), (60, 31), (60, 15)]
Escenario 7	Grafo	16384 nodos y 55285 aristas	16384 nodos y 55285 aristas
	Total path cost	42.78836441040039	42.78836441040039
	Number of expanded nodes	28382	28382
	Rango de nodos	(0,0) a (127,127)	(0,0) a (127,127)

	Coordenadas discretas	[(0, 67), (0, 121), (54, 121), (121, 107), (121, 62), (121, 30)]	[(0, 67), (0, 121), (54, 121), (121, 107), (121, 62), (121, 30)]
Escenario 8	Grafo	65536 nodos y 239428 aristas	65536 nodos y 239428 aristas
	Total path cost	134.06378173828125	134.06378173828125
	Number of expanded nodes	98457	98457
	Rango de nodos	(0,0) a (255,255)	(0,0) a (255,255)
	Coordenadas discretas	[(0, 134), (0, 243), (109, 243), (243, 215), (243, 125), (243, 61)]	[(0, 134), (0, 243), (109, 243), (243, 215), (243, 125), (243, 61)]
Escenario 9	Grafo	1048576 nodos y 3280516 aristas	1048576 nodos y 3280516 aristas
	Total path cost	906.8104248046875	906.8104248046875
	Number of expanded nodes	2265194	2265194
	Rango de nodos	(0,0) a (1023,1023)	(0,0) a (1023,1023)
	Coordenadas discretas	[(345, 819), (457, 119), (308, 685), (517, 179), (564, 360), (921, 410), (678, 740), (696, 151), (836, 794), (663, 634), (556, 758), (902, 370), (53, 435), (855, 270), (299, 718), (495, 568), (17, 569), (95, 727), (266, 1), (552, 674)]	[(345, 819), (457, 119), (308, 685), (517, 179), (564, 360), (921, 410), (678, 740), (696, 151), (836, 794), (663, 634), (556, 758), (902, 370), (53, 435), (855, 270), (299, 718), (495, 568), (17, 569), (95, 727), (266, 1), (552, 674)]
Escenario 10	Grafo	4096 nodos y 14685 aristas	4096 nodos y 14685 aristas
	Total path cost	0.07425664365291595	0.07425664365291595
	Number of expanded nodes	1926	1926
	Rango de nodos	(0,0) a (63,63)	(0,0) a (63,63)
	Coordenadas discretas	[(0, 0), (0, 42)]	[(0, 0), (0, 42)]
	Grafo	1024 nodos y 3307 aristas	1024 nodos y 3307 aristas

Uso de la IA generativa

Para ayudarnos a desarrollar el proyecto hemos usado la IA Generativa en determinados casos. En primer lugar, nos hemos ayudado del uso de la IA generativa al preguntarle el significado e interpretación de ciertos elementos para entender la práctica correctamente antes de empezar a desarrollar.

En segundo lugar, tras desarrollar los métodos del fichero [SearchEngine.py](#) le solicitamos a la inteligencia artificial que desarrollara un código con dos heurísticas aleatorias para los métodos h1 y h2. De esta forma, a pesar de que estas heurísticas no tenían nada que ver con las solicitadas en el proyecto, pudimos comprobar el correcto funcionamiento de los métodos implementados con anterioridad antes de implementar las heurísticas correctas. El resultado que nos generó la IA es el siguiente:

◆ Opción 1: Distancia Manhattan (solo movimientos en líneas rectas, sin diagonales)

```
python Copiar Editar

def h1(current_node, objective_node) -> np.float32:
    global NODES_EXPANDED
    NODES_EXPANDED += 1
    return np.float32(abs(current_node[0] - objective_node[0]) + abs(current_node[1] - objective_node[1]))
```

Esta es apropiada si solo puedes moverte arriba, abajo, izquierda y derecha — como es tu caso en `build_graph`.

◆ Opción 2: Distancia Euclídea (más precisa, aunque no siempre necesaria)

```
python Copiar Editar

def h2(current_node, objective_node) -> np.float32:
    global NODES_EXPANDED
    NODES_EXPANDED += 1
    return np.float32(np.sqrt((current_node[0] - objective_node[0])**2 + (current_node[1] - objective_node[1])**2))
```

Usa esta si quieres una heurística más "realista" en términos de distancia directa.

Por otro lado, a la hora de probar nuestro código tuvimos problemas con el método `compute_path_cost` ya que en nuestra primera versión del programa no tuvimos en cuenta que en algunos casos intentaba acceder a un nodo del grafo usando una clave de tipo string cuando en realidad debería ser una tupla. Al solicitar ayuda a ChatGPT con esto, nos modificó unas líneas de nuestro método en las que convertía la clave a tupla si era necesario. El código generado es el siguiente:

SOLUCIÓN

En tu función `compute_path_cost` (dentro de `SearchEngine.py`), asegúrate de **convertir correctamente los nodos del `solution_plan` de cadena a tupla** antes de usarlos como claves en el grafo.

✓ Arreglo sugerido para `compute_path_cost`:

```
python Copiar Editar

def compute_path_cost(G, solution_plan):
    total_cost = np.float32(0.0)
    for path in solution_plan:
        for i in range(len(path) - 1):
            node_a = eval(path[i]) if isinstance(path[i], str) else path[i]
            node_b = eval(path[i + 1]) if isinstance(path[i + 1], str) else path[i + 1]
            total_cost += np.float32(G[node_a][node_b]['weight']) # obtenemos peso de la arista
    return total_cost
```

Este cambio asegura que incluso si `solution_plan` contiene strings como `"(30, 3)"`, se convertirán a tuplas antes de acceder al grafo.



Conclusión

Con el desarrollo de este trabajo hemos podido aprender a modelar entornos mediante grafos. Por otro lado, también hemos podido aplicar heurísticas para comprobar cómo una diferente heurística puede afectar en la eficiencia de y el coste de un camino.

Por otro lado, al usar la inteligencia artificial para ayudarnos a solucionar algunos de los problemas que se han enfrentado durante la práctica hemos podido aprender a usarla de una manera más eficiente. Además, hemos aprendido la importancia que tienen los prompts y cómo puede afectar en la respuesta el uso de un prompt u otro.

En cuanto a la implementación, hemos aprendido a usar la función de *debug* de *python*. Esto nos ha sido de mucha utilidad a la hora de depurar nuestro código. Ya que al trabajar con librerías como *Numpy*, que se tratan de librerías que no hemos llegado a usar antes de la práctica, no hemos sabido usarlo muy bien. Entonces, para estos casos, la función *debug* nos ha ayudado a encontrar el fallo de manera más rápida. Ya que al no saber exactamente qué parte de nuestro código nos fallaba, con esta función hemos podido ir poco a poco comprobando el correcto funcionamiento de este.

Un problema que hemos tenido, ha sido al ejecutar los diferentes escenarios proporcionados, porque los caminos que nos salen en la gráfica no son del todo como esperábamos. Pero pensamos que esto es debido a algún error al implementar el código, ya que pensamos que la idea y el modelado del problema están bien planteados.

Por último, pensamos que la forma en la que se ha organizado el trabajo ha sido muy satisfactoria para aprender y desarrollar el proyecto. Gracias a la combinación de las clases prácticas orientadas a las dudas del proyecto y al trabajo en casa hemos podido adquirir los conocimientos necesarios para realizar el proyecto.